DHAR

Volume II — Engineering Roadmap

STAGE-BETA

“Honing Dhar — the compiler proves itself by rebuilding itself.”

Document	Stage-beta — Honing Dhar
Version	1.0
Date	September 6, 2026
Status	Ratified
Part of	The Canon of Dhar, Volume II
Depends on	Spec0, Spec1, Stage-alpha
Precedes	Stage-gamma — Optimizing Dhar

1. Mandate

Stage-beta is the first real Dhar compiler: a complete lexer, parser, type checker, and code generator, written entirely in Dhar, using only the bootstrap subset Stage-alpha can compile. Its defining test is not that it works, but that it can compile itself.

2. Scope

2.1 In scope

- The full 46-keyword language and grammar (Spec1, in full).
- Static type inference (Spec1 §4.3) and the wild dynamic mode (§4.4).
- The complete control-flow set: branching, loops, dispatch (Spec1 §8).
- mold/forge, instantiation, and the OOP/functional/logical/symbolic mechanisms (Spec1 §9–10).
- Collections: literal syntax and type names (Spec1 §11).
- The memory model: ownership, view/grab, raw memory (Spec1 §12).
- Error handling: state/trap/enforce and attempt/rescue/ensure/raise (Spec1 §13).
- Modules and visibility (Spec1 §14).
- Universal IR generation (Spec0 §4) targeting at least one backend end to end.

2.2 Out of scope

- Optimization passes of any kind — Stage-beta's code generation is expected to be direct and unoptimized. That is Stage-gamma's mandate, not this one.
- The standard library beyond what Stage-beta needs to build itself.
- Backends beyond the first one proven; additional backends are Stage-gamma's concern or later.

3. Required Architecture

Module	Responsibility
lexer.dhar	Tokenizing, keyword recognition (Spec1 §2)
parser.dhar	Recursive-descent parsing into an AST (Spec1, full grammar)
ast.dhar	AST node types — the first real use of mold for the compiler's own data (Spec1 §9)
symbol.dhar	Symbol table and scope resolution
typecheck.dhar	Type inference and checking (Spec1 §4.3–4.4)
ir.dhar	Universal IR generation (Spec0 §4)
codegen.dhar	Backend emission from the Universal IR
main.dhar	The driver — see the rule below

`main.dhar` **must** `pull` and integrate every other module into one working pipeline. A driver file that declares a pipeline but does not call into the modules that implement it is not a driver — this requirement exists specifically so that Stage-beta cannot be declared complete while its modules remain wired together only in comments.

4. Acceptance Criteria

This stage has exactly one criterion that matters, stated precisely because it is easy to convince oneself a compiler is self-hosting when it is not:

```
Let S = Stage-beta's complete source.

1. Stage-alpha compiles S           -->  E1  (generation 1)
2. E1 compiles S again              -->  E2  (generation 2)
3. compare(E1, E2) byte-for-byte, ignoring timestamps

E1 == E2   =>   Stage-beta is self-hosting.
E1 != E2   =>   Stage-beta is not complete, regardless of
                how much of the language appears to work.
```

Two supporting rules close the loopholes a partial implementation could otherwise hide behind:

1. Every construct in Spec1 must have at least one passing test compiled through this exact pipeline. A construct with no test is not considered implemented, no matter how much code exists for it.

2. A pipeline stage that prints progress messages without acting on real input — a lexer that tokenizes without a following parser consuming the tokens, or a driver that prints “Symbols: 0” regardless of the source — does not satisfy this stage's mandate even if it runs without crashing.

5. Handoff to Stage-gamma

Once self-hosting is proven, Stage-alpha **may** be archived. Stage-gamma **must not** begin until Spec1's syntax has been stable — no ratified changes — for a full development cycle, so optimization work is not built against a moving target.

Stage-beta — Honing Dhar · v1.0 · September 6, 2026 · Ratified. Revision log: v1.0 — first ratified edition.